

Unet

July 11, 2019

```
In [1]: from models import *
```

```
In [2]: from unet import *
```

0.0.1 Methods

```
In [3]: def fit(model,epoch):
```

```
    for j in tnrage(epoch,desc="Epoch",position=1,mininterval=0):
        train_loss = 0
        val_loss = 0
```

```
        one_zero_trn = 0.0
        one_zero_val = 0.0
```

```
        k = 0
```

```
        t=tqdm(iter(trn_dl), leave=False, total=len(trn_dl), miniters=0 ,postfix = f'L',
        model.train()
```

```
        for data in t:
            k+=1
```

```
            t.close()
```

```
            t.set_postfix_str(f'loss : {float(train_loss)/k} 1:1 :{one_zero_trn}',refl
            opt.zero_grad()
```

```
            data = data.type(torch.float).cuda()
            out = model(V(data))
```

```
            temp_data = data[:,0,:,:].reshape(data.shape[0],1,256,256)
            one_zero_trn = out.sum() / temp_data.sum()
```

```
            loss = f.binary_cross_entropy(out,temp_data)
            loss.backward()
            opt.step()
            train_loss += loss.data
```

```
        k = 0
```

```
        with torch.no_grad():
            model.eval()
```

```

t=tqdm(iter(val_dl), leave=False, total=len(val_dl), miniters=0, postfix = :
for data in t:
    k+=1

    data = data.type(torch.float).cuda()
    out = model(V(data))
    #print(data[:,0,:,:].reshape(16,1,256,256).shape)
    temp_data = data[:,0,:,:].reshape(data.shape[0],1,256,256)
    one_zero_val = out.sum() / temp_data.sum()
    model.one_to_one(one_zero_val)

    loss = f.binary_cross_entropy(out,temp_data)
    val_loss += loss

    t.close()
    t.set_postfix_str(f'loss : {float(val_loss)/k}    1:1 :{one_zero_val}',)

    model.record(loss)
train_loss /= len(trn_dl)
val_loss /= len(val_dl)
print('train Loss: ', train_loss.tolist(), ' validation loss: ',val_loss.t

```

0.0.2 Dataset

```

In [4]: class dataset(Dataset):
    def __init__(self,base_dir , index ):
        super().__init__()

        files = os.listdir(base_dir)
        files = files

        names = [files[i] for i in index]

        self.data = [ base_dir +i for i in names]

    def __getitem__(self, index):
        example = np.load(self.data[index]).astype(np.float)
        example = np.pad(example,(26496,26496),'constant').reshape(1,256,256)
        example = np.concatenate([example,example,example],axis = 0)
        return example

    def __len__(self):
        return len(self.data)
        raise NotImplementedError

```

0.0.3 Split Data Train/Val

```
In [5]: def split(base_dir,prc=0.2 , dataset_pct = 1):

        total_size = int(len(os.listdir(base_dir)) * dataset_pct)

        val_size = int(total_size*prc)

        trn_index = np.arange(total_size - val_size)
        val_index = np.arange(trn_index[-1]+1,total_size)

        val = dataset(base_dir , val_index)
        trn = dataset(base_dir , trn_index)
        return trn , val

In [6]: trn , val = split('../..//dataset/',dataset_pct=0.5)
```

0.0.4 Prepare DataLoaders

```
In [7]: trn_dl = DataLoader(trn,batch_size=16,num_workers=2,shuffle=False)
        val_dl = DataLoader(val,batch_size=16,num_workers=2,shuffle=False)
```

0.1 Model

```
In [8]: class U(nn.Module):
        def __init__(self):
            super().__init__()
            self.loss = []
            self.one = []
            ##Load pre-trained
            self.unet = UNet().cuda()
            state_dict = torch.load('weights/unet.pt', map_location="cuda:0")
            self.unet.load_state_dict(state_dict)
        def forward(self,inputs):
            k = self.unet(inputs)
            return k

        def record(self,record):
            self.loss.append(record)

        def one_to_one(self,record):
            self.one.append(record)

In [9]: m = U().cuda()

In [10]: opt = torch.optim.Adam(m.parameters(),1e-4)

In [11]: fit(m,1)
```

```
HBox(children=(IntProgress(value=0, description='Epoch', max=1, style=ProgressStyle(description

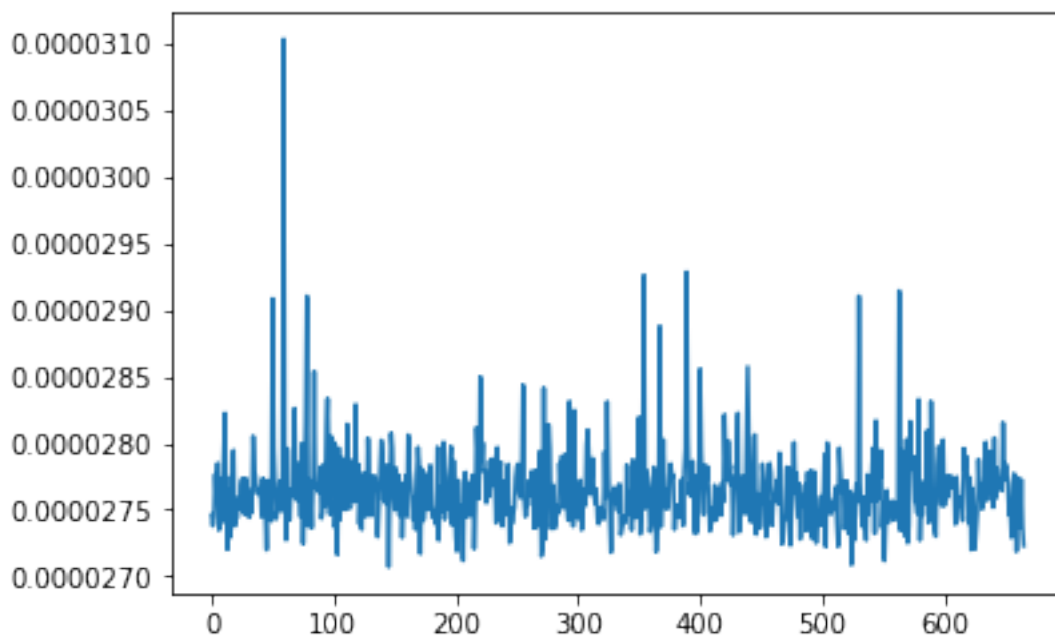
100%|| 665/666 [01:37<00:00, 7.09it/s, loss : 2.7655759700258873e-05 1:1 :1.006230473518371

train Loss: 0.0007786486530676484 validation loss: 2.7655121812131256e-05
```

The image looks noisy because loss is recorded against minibatches. The very small values of loss are clear.

```
In [12]: plt.plot(m.loss)
```

```
Out[12]: [<matplotlib.lines.Line2D at 0x7ff9e0ee3550>]
```



0.2 This meter is called 1:1

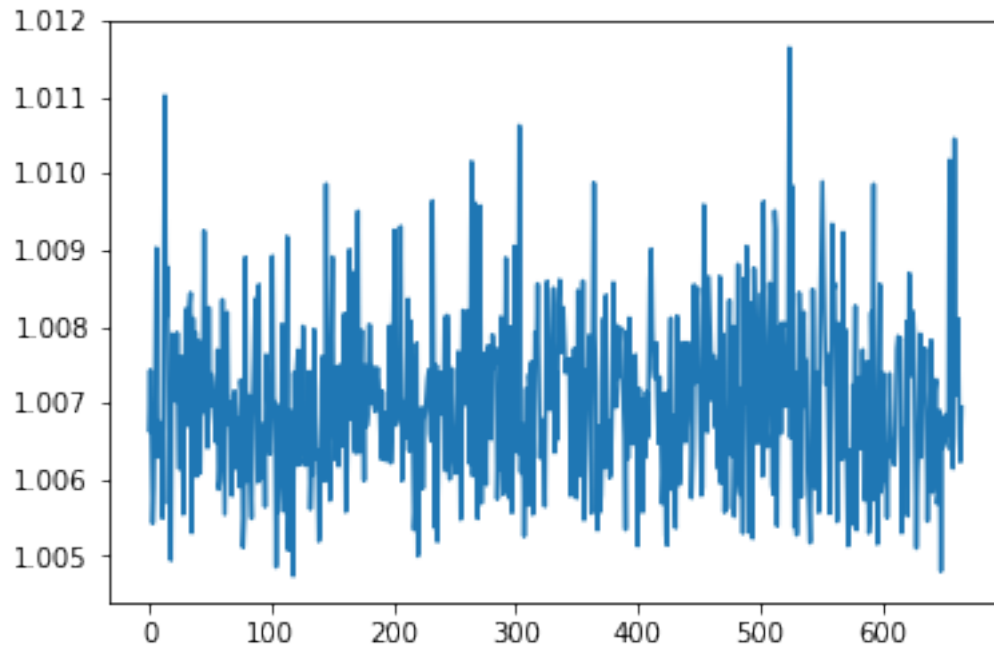
Since most of the input is zero due to

- 1- conversion from hex to binary
- 2- Padding to fit the shape

It was thought that the model would predict all the output to be zero and still get low loss. So the meter was watching the ratio of the count of ones in input and in output. The average in the image is around one, which proves that the model is moving in the right direction.

```
In [13]: plt.plot(m.one)
```

```
Out[13]: [<matplotlib.lines.Line2D at 0x7ff9e0e83748>]
```



```
In [16]: np.save('records/unet_half.npy',m.loss)
```

0.3 From scratch

```
In [24]: class U(nn.Module):
    def __init__(self):
        super().__init__()
        self.loss = []
        self.one = []
        ##Load pre-trained
        self.unet = UNet().cuda()

    def forward(self,inputs):
        k = self.unet(inputs)

        return k
    def record(self,record):
        self.loss.append(record)
    def one_to_one(self,record):
        self.one.append(record)

In [25]: m = U().cuda()
         opt = torch.optim.Adam(m.parameters(),1e-4)

In [26]: fit(m,1)
```

```
HBox(children=(IntProgress(value=0, description='Epoch', max=1, style=ProgressStyle(description
```

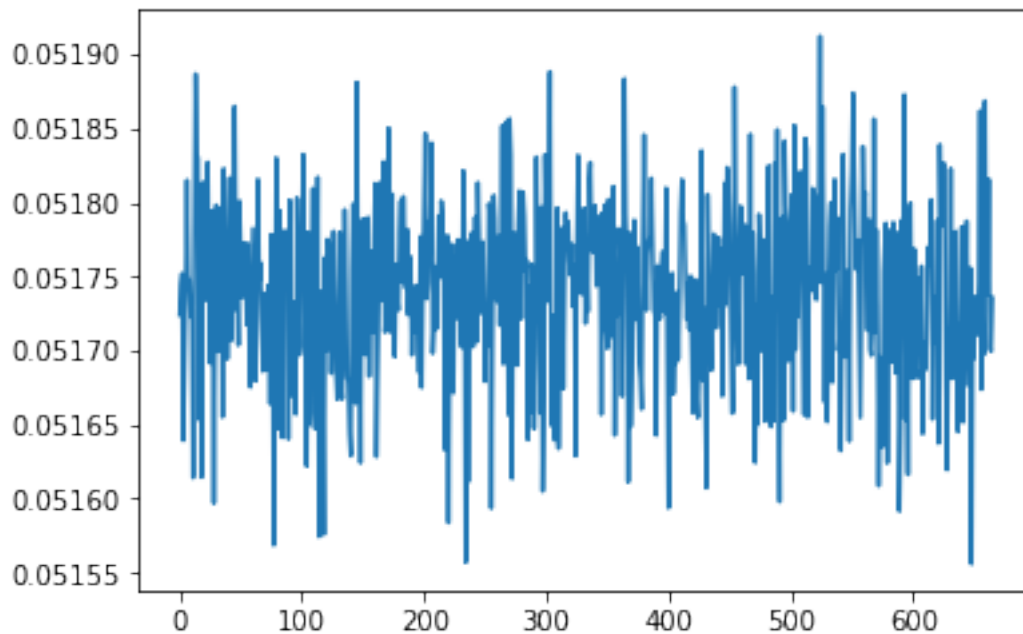
```
100%|| 666/666 [01:08<00:00, 10.44it/s, loss : 0.05173904831345017    1:1 :13.8721342086792]
```

```
train Loss:  0.21991844475269318    validation loss:  0.05173904821276665
```

The average of the loss is relatively low but still much higher than the pre-trained model but it had decreasing trend

```
In [27]: plt.plot(m.loss)
```

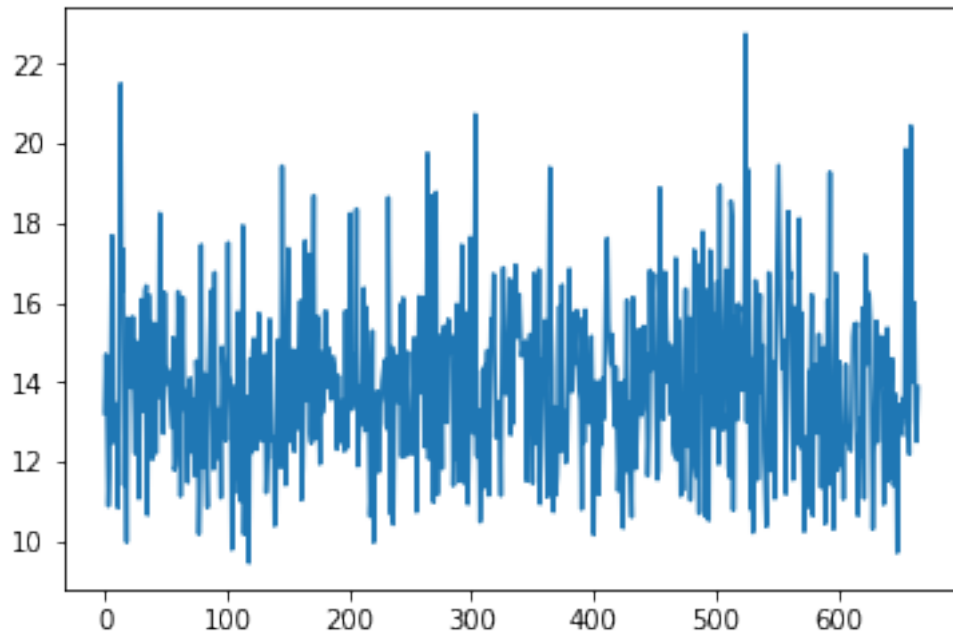
```
Out[27]: [<matplotlib.lines.Line2D at 0x7ff9df3b3908>]
```



Much higher 1:1 ratio but it had a decreasing trend

```
In [31]: plt.plot(m.one)
```

```
Out[31]: [<matplotlib.lines.Line2D at 0x7ff9e0be3048>]
```



```
In [28]: np.save('records/unet_scratch_half.npy',m.loss)
```

0.4 Comparisons

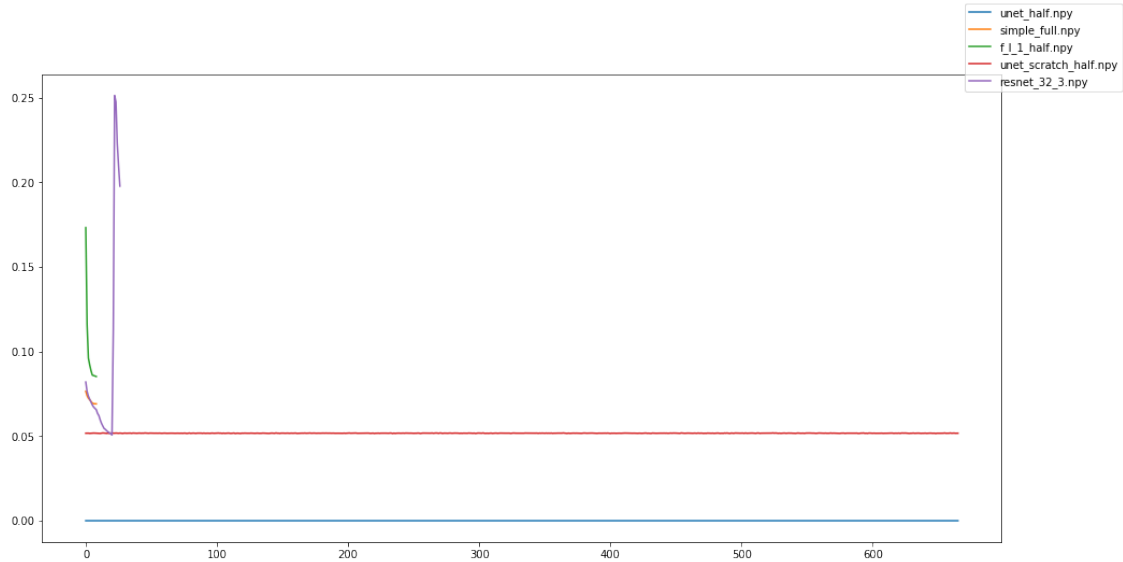
We padded the input to the recommended dimensions 256x256

```
In [29]: a = os.listdir('records/')

```

```
In [30]: fig , ax = plt.subplots(figsize=(16,8))
         for i in a :
             b = np.load('records/'+i)
             ax.plot(b, label=i)
         fig.legend()
```

```
Out[30]: <matplotlib.legend.Legend at 0x7ff9e0433978>
```

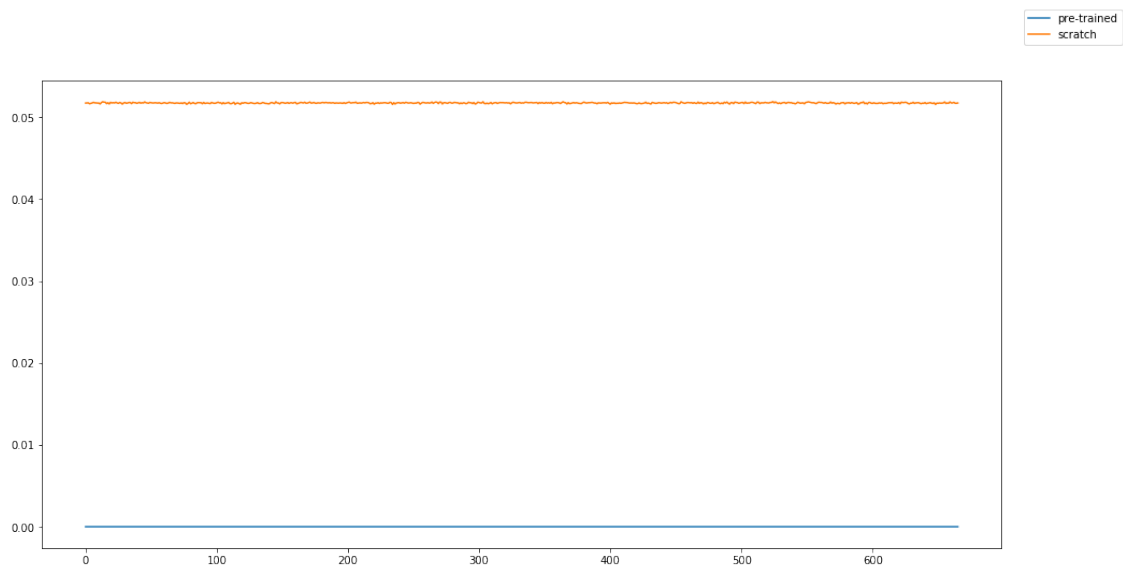


Clearly the pre-trained unet Model outperforms any other implementation

The difference in lengths is due to loss in u-net was recorded for mini-batches, while others were recorded for epochs. Note: U-Net, simple had the change to run over the whole dataset while the Resnet accuracy was lost by a smaller subset of data.

```
In [34]: fig, ax = plt.subplots(figsize=(16,8))
         ax.plot(np.load('records/unet_half.npy'), label = 'pre-trained')
         ax.plot(m.loss, label = 'scratch')
         fig.legend()
```

Out [34]: <matplotlib.legend.Legend at 0x7ff9d0003a58>



Pre-trained weights crucially affects the speed and values of loss